

Katholieke
Universiteit
Leuven

**Department of
Computer Science**

SESSION 2

Race Conditions

Groothuis Thomas (r0955783)
De Smet Feniks (r0891378)

Academic year 2025–2026

Contents

1 Problem & Goals **2**

2 Methodology **2**
2.1 Strategy . . . 2
2.2 Implementation . . . 2
2.2.1 Method 1: Using a for-loop on the CPU . . . 2
2.2.2 Method 2: Using a GPU kernel employing atomic operation . . . 3
2.2.3 Method 3: Using a GPU kernel with reduction . . . 3

3 Results **4**

A CUDA Implementation **4**

B Data Collection Script **7**

1 Problem & Goals

This lab addresses the problem of finding the maximum value in a 1D array. To achieve this, three different methods are implemented and compared:

- A for-loop on the host(CPU)
- A GPU kernel employing atomic operations
- A GPU kernel utilizing parallel reduction

Besides this we investigate the relationship between the execution time and the total length of the input array.

2 Methodology

2.1 Strategy

When dealing with parallel execution on the GPU, fundamental hardware limitations and thread management must be considered.

First, there is a maximum of 1024 threads per block. As a result a single block can only compare up to 2048 elements (2×1024). This means that we need to work with multiple blocks if the input size goes higher than 2048. It would also require us to implement additional synchronization logic. Because blocks cannot synchronize with each other directly, we use the host (CPU) to manage this global synchronization.

Second, we must ensure that all elements are processed, even for massive arrays. A theoretical constraint could be calling the kernel with fewer threads than data elements (e.g., one tenth). In our implementation, we avoid this problem entirely: our strategy dynamically scales the grid size rather than relying on a fixed number of threads. By using the formula `(N + threadsPerBlock - 1) / threadsPerBlock`, the host ensures that the total number of launched threads always meets or slightly exceeds the number of data elements N. This guarantees that every data element is processed by exactly one thread, while any remaining threads in the final block are safely kept idle using a boundary check (`if (idx < n)`).

Although not implemented in our current code, if we were hypothetically constrained to launch fewer threads than there are elements, this 1-to-1 mapping approach would leave a large portion of the data unprocessed. To resolve this limitation without relying on advanced thread-striding (see next lab), the workload would need to be partially serialized. This could be achieved either by having the host process the array in chunks (launching the kernel iteratively for each segment) or by introducing a sequential loop inside the kernel itself. This would allow each individual thread to process a localized subset of elements before participating in the block-wide operations.

2.2 Implementation

Similar to the first lab session, the program begins by verifying the command-line arguments. It expects exactly one argument `N`, which defines the length of the array.

The argument `N` is then used to allocate the array in the host memory. To populate the array with test data, we implemented a custom randomization process using a Mersenne Twister pseudo-random number generator (PRNG). Initially, each element is assigned a value equal to its index. During this initialization, each element has a 10% probability of being artificially boosted by adding `N` to its base value. The array is also completely shuffled to ensure a random distribution of these peak values. While not explicitly required for this task, we note in hindsight that offloading this initialization and shuffling process to a custom GPU kernel could have reduced the data generation overhead. But for the goals of the lab this CPU-based random array generation suffices.

The main program executes the three different max-detection methods sequentially. We use `cudaEvent` to record each methods execution time.

2.2.1 Method 1: Using a for-loop on the CPU

In this sequential method the host initializes the maximum value using the first element of the array. A standard loop then iterates through the remaining elements. The stored maximum is updated whenever the loop encounters a greater value.

2.2.2 Method 2: Using a GPU kernel employing atomic operation

Since this method utilizes the GPU, unlike the first approach, memory is allocated in device DRAM for both the input array and the variable storing the maximum value. The array is then copied from the host to the device and the device maximum value is initialized to -1.

To maximize parallelism we configure the execution layout with 1024 threads per block. As a result the grid is scheduled with `(N + threadsPerBlock - 1) / threadsPerBlock` blocks. The `findMaxAtomic` kernel is subsequently launched using this grid configuration. It takes pointers to the device array and the device maximum value alongside the array length `N` as arguments.

Inside the kernel each thread calculates its unique global index using the formula `idx = threadIdx.x + blockIdx.x * blockDim.x`. A boundary check ensures that only threads with an index strictly lower than `N` perform actual computations. These active threads execute an `atomicMax` operation. This function safely compares the current value stored at the `d_max` address with the thread's respective array element and stores the greater value back to memory at the same address in one atomic transaction.

Following the kernel execution and result retrieval the allocated device memory is safely freed.

2.2.3 Method 3: Using a GPU kernel with reduction

Similar to the second method, we begin by allocating the input array `d_in` in the device memory. Additionally, we allocate an output array, `d_out`, which is used to store the intermediate reduction results. The size of this output array is calculated using the formula `((N + threadsPerBlock - 1) / threadsPerBlock) * sizeof(int)`, ensuring it has exactly enough capacity to hold one maximum value per scheduled thread block. We also initialize a `numElements` variable to track the remaining number of elements that need processing.

The parallel reduction algorithm operates on two distinct levels: an external loop managed by the host (CPU) and an internal loop executed within each individual thread block (GPU).

The core host-side logic relies on an external `while` loop that acts as the global, grid-level synchronizer. As long as `numElements` is strictly greater than one, this outer loop dynamically calculates the required number of blocks for the current global iteration and launches the `blockMaxReduction` kernel.

Once the kernel is launched, the internal block-level reduction takes over. First, each thread calculates its unique global index using the same formula as method 2. After this initialization, the kernel enters its own internal `for` loop to perform multiple local reduction rounds within the isolated block. Using a `stride` variable that initializes at 1 and doubles each round, active threads calculate the maximum of two paired elements. The winning maximum is written directly back into the original input array.

As this internal loop progresses, half of the previously active threads become idle. To prevent race conditions during these local rounds, a `__syncthreads()` barrier is strictly enforced, ensuring all threads within the block complete their memory writes before the block's next internal round begins. Once the stride exceeds the block size (`blockDim.x`), the block's internal reduction is finished. The absolute maximum value of that specific block is safely consolidated, and thread 0 writes this local maximum to its designated position in the `d_out` array.

Because each block operates independently and produces exactly one local maximum, a single kernel execution results in a new, smaller array of intermediate maximums. This marks the end of one global round. The total number of remaining elements now equals the number of blocks scheduled in the grid, so the `numElements` variable is updated accordingly.

To efficiently prepare for the next iteration of the host's outer reduction loop, a pointer swap is performed. A temporary pointer (`tmp`) stores the memory address of `d_in`, after which `d_in` is updated to point to `d_out`. This means the newly calculated block maximums become the input data for the next global round. Finally, `d_out` is assigned the address stored in `tmp`. This ping-pong buffering strategically recycles the device memory, completely avoiding costly memory allocations (`cudaMalloc`) during the outer loop and significantly improving performance.

Once the external `while` loop terminates, the grid-level reduction has condensed all block-level results into a single global maximum. This final value is then copied from `d_in` back to host memory, after which all device memory is safely released using `cudaFree`.

3 Results

The results from the GPU are correct because they were compared with the CPU methods. We executed all array sizes 20 times to ensure stable results. The graph shows the average across these 20 runs. Furthermore, for every run, we verified that both GPU methods successfully returned the exact same maximum value as the sequential CPU method, confirming the correctness of our implementations.

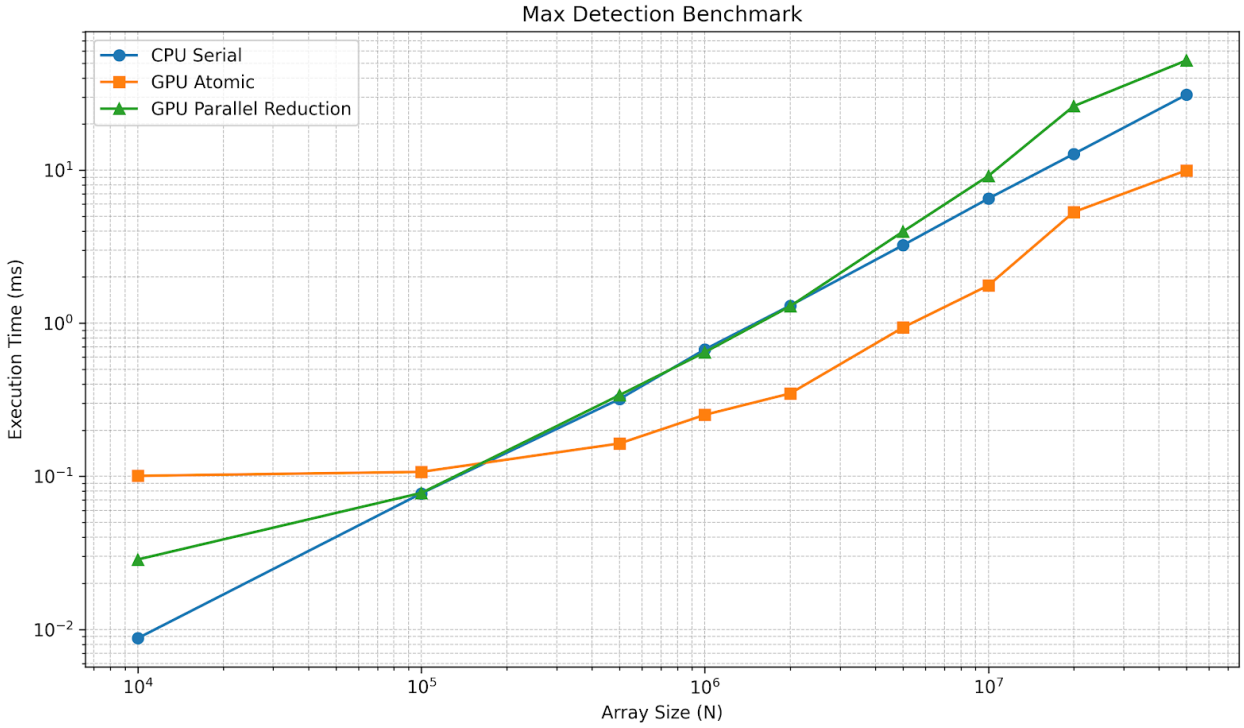


Figure 1: Max Detection Benchmark results on NVIDIA T400 4GB

CPU time is linear because the CPU method compares each element with the found maximum. It scales with O(N) as it needs to check every element sequentially.

The GPU kernel with atomic operations works faster than the CPU. A leading factor for this divergence is the addition of the L2 cache and the optimization of that cache over the years. While atomic operations inherently serialize memory access, modern NVIDIA architectures resolve global atomic operations directly within the GPU's L2 cache via dedicated hardware units. This bypasses the high latency of the global DRAM memory. As officially stated by NVIDIA: "Thanks to a combination of more atomic units in hardware and the addition of the L2 cache, atomic operations performance is up to 20× faster in Fermi compared to the GT200 generation" [1].

The GPU kernel with reduction should theoretically be O(log N). However, the results provide a less favourable outcome. Because the kernel repeatedly updates the `d_in` array, it is forced to perform frequent read and write operations directly in the slow global DRAM. We conclude that the reduction optimization does not outperform the slow DRAM (therefore we could have used shared memory, but this was not the scope of the lab).

Now the question remains: could this be implemented for a 2D array, and if so, how? The answer is yes.

A 2D array could be handled by simply flattening it into a single, longer 1D array. Alternatively, we could use a multi-phase approach where each row of the 2D matrix is processed individually like a 1D array, resulting in a maximum value for each row. These row maximums are then collected into a new array and processed one final time to find the global maximum. Both the atomic and reduction GPU kernel methods could be used for either approach.

A CUDA Implementation

```cpp
#include <iostream>
#include <cuda_runtime.h>
#include <random>
#include <cstdlib>

// ---------------- Atomic max kernel ----------------
__global__ void findMaxAtomic(int *d_a, int *d_max, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(d_max, d_a[idx]);
    }
}

// ---------------- Block-level reduction kernel----------------
__global__ void blockMaxReduction(int *d_in, int *d_out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if ((threadIdx.x % (2 * stride)) == 0) {
            int neighbour = i + stride;

            if (neighbour < n) {
                d_in[i] = max(d_in[i], d_in[neighbour]);
            }
        }
        __syncthreads();
    }

    if (threadIdx.x == 0 && i < n) {
        d_out[blockIdx.x] = d_in[i];
    }
}


int main(int argc, char* argv[]) {
    if (argc != 2) {
        std::cout << "Usage: ./session2 <array_size>\n";
        return 1;
    }

    int N = atoi(argv[1]);
    int *a = new int[N];

    std::mt19937 prng(std::random_device{}());
    std::uniform_real_distribution<double> probDist(0.0, 1.0);

    double boostChance = 0.1;

    for (int i = 0; i < N; i++) {
        a[i] = i;
        if (probDist(prng) < boostChance) {
            a[i] += N;
        }
    }

    for (int i = N - 1; i > 0; i--) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(prng);
        std::swap(a[i], a[j]);
    }

    // ---------------- CPU ----------------
    cudaEvent_t startCPU, stopCPU;
    cudaEventCreate(&startCPU);
    cudaEventCreate(&stopCPU);

    cudaEventRecord(startCPU);
    int maxValue = a[0];
    for (int i = 1; i < N; i++) {
        if (a[i] > maxValue) {
            maxValue = a[i];
        }
    }
    cudaEventRecord(stopCPU);
```

```
    cudaEventSynchronize(stopCPU);

    float cpuTime = 0.0f;
    cudaEventElapsedTime(&cpuTime, startCPU, stopCPU);
    std::cout << "CPU serial max: " << maxValue << " (time: " << cpuTime << " ms)" << std::
    endl;

    cudaEventDestroy(startCPU);
    cudaEventDestroy(stopCPU);

    // ---------------- GPU atomic max ----------------
    int *d_a, *d_max;
    cudaMalloc(&d_a, N * sizeof(int));
    cudaMalloc(&d_max, sizeof(int));

    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    int initMax = -1;
    cudaMemcpy(d_max, &initMax, sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 1024;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEvent_t startGPU, stopGPU;
    cudaEventCreate(&startGPU);
    cudaEventCreate(&stopGPU);
    cudaEventRecord(startGPU);

    findMaxAtomic<<<blocks, threadsPerBlock>>>(d_a, d_max, N);
    cudaDeviceSynchronize();

    cudaEventRecord(stopGPU);
    cudaEventSynchronize(stopGPU);

    float gpuTimeAtomic = 0.0f;
    cudaEventElapsedTime(&gpuTimeAtomic, startGPU, stopGPU);

    int maxAtomic;
    cudaMemcpy(&maxAtomic, d_max, sizeof(int), cudaMemcpyDeviceToHost);

    std::cout << "GPU atomic max: " << maxAtomic << " (time: " << gpuTimeAtomic << " ms)" <<
    std::endl;

    cudaFree(d_max);
    cudaEventDestroy(startGPU);
    cudaEventDestroy(stopGPU);

    // ---------------- GPU reduction max ----------------
    cudaMalloc(&d_a, N * sizeof(int));
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    int numElements = N;
    int *d_in = d_a;
    int *d_out;
    cudaMalloc(&d_out, ((N + threadsPerBlock - 1)/threadsPerBlock) * sizeof(int));

    cudaEvent_t startRed, stopRed;
    cudaEventCreate(&startRed);
    cudaEventCreate(&stopRed);
    cudaEventRecord(startRed);

    while (numElements > 1) {
        blocks = (numElements + threadsPerBlock - 1) / threadsPerBlock;
        blockMaxReduction<<<blocks, threadsPerBlock>>>(d_in, d_out, numElements);
        cudaDeviceSynchronize();

        numElements = blocks;

        int *tmp = d_in;
        d_in = d_out;
        d_out = tmp;
    }

```

```
    cudaEventRecord(stopRed);
    cudaEventSynchronize(stopRed);

    int maxReduction;
    cudaMemcpy(&maxReduction, d_in, sizeof(int), cudaMemcpyDeviceToHost);

    float gpuTimeReduction = 0.0f;
    cudaEventElapsedTime(&gpuTimeReduction, startRed, stopRed);

    std::cout << "GPU reduction max: " << maxReduction << " (time: " << gpuTimeReduction << "
     ms)" << std::endl;

    cudaFree(d_a);
    cudaFree(d_out);
    cudaEventDestroy(startRed);
    cudaEventDestroy(stopRed);

    delete[] a;
    return 0;
}
```

B Data Collection Script

To automate the benchmarking process, a Python script was used to execute the compiled CUDA binary for various array sizes. For each size N, the program was executed 20 times to obtain a statistically significant average. The output of each run was parsed using regular expressions and stored in a structured format for further analysis.

```
import subprocess
import csv
import matplotlib.pyplot as plt

sizes = [
10_000, 100_000, 500_000, 1_000_000, 2_000_000,
5_000_000, 10_000_000, 20_000_000, 50_000_000
]

runs_per_size = 20

csv_file = "results.csv"

cpu_times = []
atomic_times = []
reduction_times = []

for N in sizes:
    print(f"Running N = {N}")

    cpu_total = 0
    atomic_total = 0
    reduction_total = 0

    for _ in range(runs_per_size):
        result = subprocess.run(["./session2", str(N)],capture_output=True,text=True)
        output = result.stdout.strip()
        parts = output.split()

        if len(parts) < 4:
            print(f"Warning: unexpected output: {output}")
            continue

        cpu_total += float(parts[1])
        atomic_total += float(parts[2])
        reduction_total += float(parts[3])

    cpu_avg = cpu_total / runs_per_size
    atomic_avg = atomic_total / runs_per_size
    reduction_avg = reduction_total / runs_per_size

    cpu_times.append(cpu_avg)
```

```
    atomic_times.append(atomic_avg)
    reduction_times.append(reduction_avg)

    print(f"N={N} | CPU={cpu_avg:.2f} ms | Atomic={atomic_avg:.2f} ms | Reduction={
    reduction_avg:.2f} ms")

with open(csv_file, mode='w', newline='') as f:
writer = csv.writer(f)
writer.writerow(["N", "CPU_ms", "GPU_Atomic_ms", "GPU_Reduction_ms"])
for i, N in enumerate(sizes):
    writer.writerow([N, cpu_times[i], atomic_times[i], reduction_times[i]])

print(f"Results saved to {csv_file}")
```

Listing 1: Automated Python benchmark script

References

[1] NVIDIA Corporation, "NVIDIA's Next Generation CUDA Compute Architecture: Fermi," NVIDIA Whitepaper, 2009. [Online]. Available: https://www.nvidia.com/content/pdf/fermi_white_papers/nvidia_fermi_compute_architecture_whitepaper.pdf